How Weight Tying Elicits Iterative Computation in Looped Transformers

Anonymous authors
Paper under double-blind review

Abstract

Looped transformers apply the same block of layers many times to each input, and have been shown to learn iterative solutions that progress roughly one step per iteration (a *staircase*). It remains unclear, however, how exactly weight tying leads to such solutions. We investigate this using *directed permutation traversal*, a task in which a model must report the node reached after H hops from a starting node. Its precisely defined intermediate states let us track the model's progress iteration by iteration and layer by layer using linear probes and causal interventions. We find that the underlying mechanism is gradient pooling, the summing of gradients across repeated uses of the shared block. We test this by giving each iteration its own copy of the block and varying how much of each copy's update comes from the pooled gradient. With more pooling, the task is learned more quickly and reliably, and the solution moves from the layout of untied models, which expose intermediate states only near the output, to the staircase. Conversely, training a looped model on only one iteration's gradient, however it is scaled or sampled, prevents the solution from forming. Pooling thus favours a reusable hop operation which, once formed, is composed across iterations, explaining both the rapid acquisition of multi-hop capability during training and the staircase at inference. Together, these findings provide a mechanistic account of how weight tying elicits iterative computation, informing design choices as looped models scale.

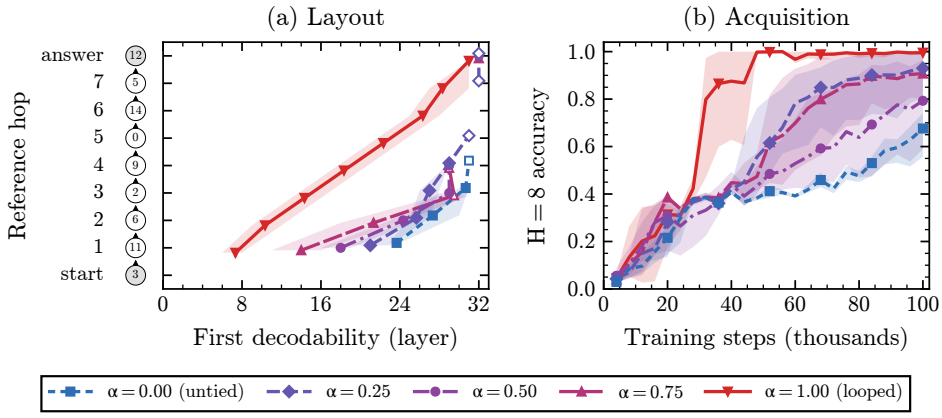


Figure 1: **Gradient pooling elicits iterative solutions.** We interpolate between an untied model ($\alpha = 0$), whose blocks each update from their own gradient, and a looped model ($\alpha = 1$), whose blocks all share one pooled update, so α is the share of each block's update taken from the pooled gradient. (a) First decodability of each reference hop at $H = 8$. (b) Last-query accuracy at $H = 8$ over training. Full details in Section 4.1.

1 Introduction

Looped transformers are often touted for their *iterative inductive bias*, a propensity to learn solutions that involve repeated application of the same transformation (Gao et al., 2025; Saunshi et al., 2024;

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

2025). But the exact mechanism by which such solutions result from weight tying remains elusive. Understanding it has become imperative now that looped models reach tens of billions of parameters (Wang et al., 2026b; Wu et al., 2025; Yang et al., 2026), and rival their untied counterparts (Zhu et al., 2025). To understand how looping puts depth to work, we follow the solution as it forms.

By virtue of weight tying, the hidden states of successive iterations are directly comparable. Their sequence traces a trajectory whose geometry has proven important for ensuring convergence and training stability (Blayney et al., 2026; Prairie et al., 2026), and for motivating architectural augmentations (Pappone et al., 2025). Yet, a trajectory alone does not reveal what the model is actually computing. To see that, the computation must be read against the task's reference states, i.e., the intermediate answers that a correct solution passes through. Recent work reads looped models this way and finds them advancing one reference state at a time (Kohli et al., 2026; Sesodia et al., 2026; Zhang et al., 2026).[1] Here, we show that neither the task nor the looped architecture forces this stepwise progress, and ask what during training produces it.

We investigate this using *directed permutation traversal* (DEPO), a synthetic task in which a model receives a shuffled table of directed edges that form a cycle and must report the node reached after H hops from a given starting node (Figure 2). Each hop defines a reference state, allowing us to follow progress inside the model without supplying an intermediate answer trace during training. Sesodia et al. (2026) found on this task that looped models expose reference states along a gradual staircase, progressing roughly one hop per iteration, at a learning rate at which untied models of matched computational depth fail. At a lower learning rate, untied models solve the task too but expose the reference states mainly in their final layers (Sesodia et al., 2026; Figure 3), and we find that a looped model trained without the hop curriculum solves it with no forward staircase (§4).

To understand what in training produces the staircase, we examine how weight tying affects the learning signal. Giving each iteration its own copy of the block and varying how much of each copy's update comes from the pooled gradient, we find that more pooling makes the solution more likely and earlier, and moves it from the untied layout to the staircase (Figure 1). Together with interventions on the looped model's gradient, this paints a picture in which pooling builds a reusable hop operator, which the forward pass then composes across iterations to produce the staircase. We track its emergence during training, examine the shared block's internal computation, and test how the arrangement of parameter reuse shapes learning. Together, these findings give the iterative inductive bias a mechanistic basis, informing the design of sharing patterns in parameter-efficient models (Lan et al., 2020).

Contributions.

1. **Gradient pooling favours the learning of a reusable one-hop operator.** Varying how much of each update is pooled across iterations sets how readily the solution is learned and where in depth it is built, and interventions that disrupt pooling while retaining the tied forward architecture impair multi-hop acquisition (Figure 1, §4.1, §4.2).
2. **The reusable operator explains the staircase and its rapid acquisition.** Composing the one-hop operator across iterations exposes one more intermediate state per iteration at inference and, once the operator forms during training, enables the rapid acquisition of longer traversals, a *compositional burst* that we predict and observe (§4.3).
3. **The benefits of pooled learning depend on how reuse is arranged.** We show that cyclic and contiguous sharing produce different acquisition speeds for multi-hop solutions at matched depth, parameter count and uses per group (§4.4).

2 Related Work

Looped transformers. Originally referred to as Universal Transformers (Dehghani et al., 2019), looped, or depth-recurrent, transformers have entered a renaissance, following seminal proofs of concept at the billion-parameter scale (Geiping et al., 2025; McLeish et al., 2026; Zhu et al., 2025). These have motivated scaling to tens of billions of parameters (Gao et al., 2026; Wu et al., 2025; Yang et al., 2026), while numerous variants explored when, how, and how much to loop (Bae et al., 2025;

[1] Most language-modelling tasks provide no such reference states.

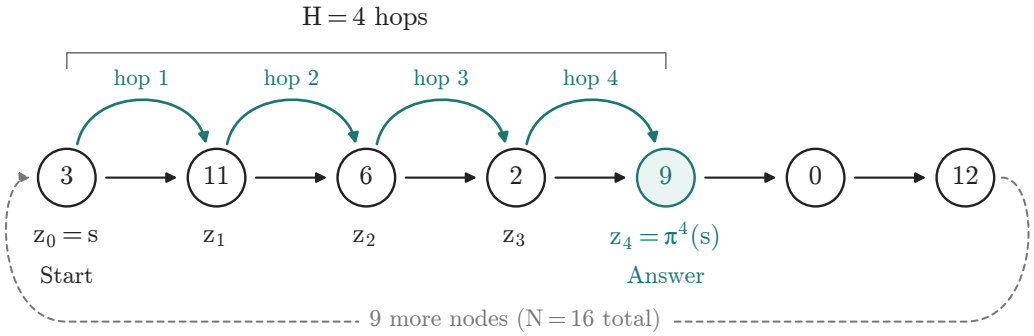


Figure 2: **Directed permutation traversal.** Given a shuffled edge list, the model must perform H sequential hops from a starting node s and report where it lands. Here, $s = 3$ and the $H = 4$ hops take the model through reference states $z_1 = 11$, $z_2 = 6$ and $z_3 = 2$, before reaching the answer $z_H = 9$. Curved arrows mark the hops and straight arrows mark the edges, and the dashed segment represents the nine omitted nodes that complete the $N = 16$ cycle.

Han, 2026; Heo et al., 2025; Jeddi et al., 2026; Knupp et al., 2026; Wang et al., 2025; Yu et al., 2026). Meanwhile, scaling laws find that looped models are compute-optimal over untied models (Prairie et al., 2026), even though one extra loop is worth less than one extra unique block (Schwethelm et al., 2026). Theoretically, looping provides, innately, the depth, state tracking and programmability that fixed-depth models must rely on chain of thought for (Giannou et al., 2023; Merrill & Sabharwal, 2025a;b; Mozer et al., 2026; Xu & Sato, 2026). This allows them to match deeper untied models with a fraction of the parameters (Saunshi et al., 2025), generalise to longer inputs (Fan et al., 2025), and learn iterative algorithms (Yang et al., 2024).

Mechanistic understanding of looping. Most mechanistic analyses treat the successive hidden states of a looped model as the trajectory of a dynamical system (Bai et al., 2019), along which trained models trace orbits and drifts (Geiping et al., 2025; Pappone et al., 2025) before settling into a cyclic fixed point (Blayney et al., 2026). The same lens traces residual explosion to the spectral norm of the input injection (Prairie et al., 2026) and, in concurrent work, to forward updates that align across iterations (Wang et al., 2026a) and a shared parameter update that sums gradients across them (Li et al., 2026). Additionally, Gong et al. (2025) appeal to loss-landscape geometry to explain the training advantage of looped models, while Kaissis et al. (2026) decompose the summed gradient by iteration for data attribution. None of these analyses, however, reads the trajectory against a reference solution, and those that note the summed update use it to set residual scaling or to attribute influence, leaving open how it shapes the solution that training finds.

Going inside the loop. Where a task has known intermediate answers, the hidden states of a loop can be read against them. Kohli et al. (2026) do so for multi-hop question answering that requires composing facts learned in training. The pointer-chasing task, however, provides all the required information in the prompt, disentangling memorisation from multi-hop reasoning. On this task, Sesodia et al. (2026) found that looped models expose the intermediate nodes along a staircase, roughly one hop per iteration, while untied models of the same depth fail to solve it at the same learning rate. Untied models solve it only at a lower learning rate, and then expose the nodes only in their final layers. Concurrently, Zhang et al. (2026) supervise every prefix of a group word problem and show that tying switches the learned algorithm from the parallel scan of standard transformers to a serial frontier, which they attribute to an architecture prior. We deliberately supervise no intermediate state, so any structure in the solution is chosen by training rather than imposed by the targets.

3 METHODS

Task. We use directed permutation traversal (DEPO), a pointer-chasing task from the Physics of Language Models lineage (Allen-Zhu, 2025) (Figure 2). The model is presented the edges $v \to \pi(v)$

of a single directed cycle on $N = 16$ nodes, in shuffled order, and must return the node $\pi^H(s)$ reached after H hops from a start node s. Since the task is strictly sequential, a correct traversal passes through the intermediate reference states $z_j = \pi^j(s)$, $j \in \{1, \ldots, H-1\}$, which we probe for and intervene on. Because the nodes form one cycle, H forward hops coincide with $16 - H$ backward hops, so $H = 8$ is the hardest difficulty we evaluate at.

Models and training. We define an $L \times T$ looped model as one which applies a block of L distinct layers T times, and call each pass through the block an *iteration*. Writing $h^{(0)}$ for the token embeddings and f_θ for a block of L transformer layers with parameters θ, the two families compute

$$\text{looped:} \quad h^{(t)} = f_\theta\big(h^{(t-1)} + \tau_t\big), \tag{1}$$

$$\text{untied:} \quad h^{(t)} = f_{\theta_t}\big(h^{(t-1)}\big), \tag{2}$$

for $t = 1, \ldots, T$, where τ_t is a learned iteration embedding. The looped model reuses one parameter set θ at every iteration, whereas the untied model has a distinct θ_t at each of its T block positions, hence LT distinct layers in total. The iteration embedding follows prior looped architectures (Dehghani et al., 2019; Xu & Sato, 2025; Zhu et al., 2025), and the staircase forms without it (Table B.3).

We train 8×4 (Figure B.3), 4×8 and 2×16 loops from scratch at a fixed effective depth $LT = 32$. Unless stated otherwise, looped results refer to the 4×8 model and untied results to 32 distinct layers. Both families use residual width $d = 256$, eight attention heads, and a next-token cross-entropy loss over the whole sequence from a single readout at the final layer. We use no intermediate readouts (Lee et al., 2015), adaptive halting (Banino et al., 2021; Graves, 2016), or input re-injection, so that depth, counted in completed layer evaluations, is a consistent yardstick. Because of weight tying, the two are not matched in parameter count (3.22M against 25.72M) or in initialisation (Appendix A.2). The exception is the shared-initialisation cohort of §4.3 and §4.4, in which every arm of a seed starts from copies of one looped block and so computes the same function at step 0 (Table A.3). The task, architectures and curriculum follow Sesodia et al. (2026). All models train for 100k steps under one recipe at peak learning rate $\eta = 3 \times 10^{-4}$ unless stated otherwise (Appendix A.3), with a curriculum that holds the hop range at $H = 1$ for the first 7k steps and widens it linearly to $H \leq 15$ by 30k steps.

Linear probes and metrics. To ask whether a reference state is present at a given depth, we fit a separate affine 16-way probe for every depth and hop j on the residual state at the token preceding the answer, using 4,000 fitting and 1,000 held-out prompts. We define the *selectivity* of hop j at depth ℓ as the probe's held-out accuracy minus that of a control probe trained on shuffled labels (Hewitt & Liang, 2019), $S_{\ell,j} = \mathrm{Acc}_{\mathrm{real}}(\ell, j) - \mathrm{Acc}_{\mathrm{shuffled}}(\ell, j)$, which is near one when the node can be read out linearly and near zero when it cannot. We say that hop j is *decodable* at depth ℓ when $S_{\ell,j}$ reaches 0.5, and its *onset* is the first such depth at which this happens (Appendix A.5). To watch these onsets form during training, we repeat the fit on checkpoints saved every 4k steps (§4.3). Each prompt contains five consecutive queries over one edge list (permutation). We use *first-query accuracy*, scored on the first of the five queries, as the primary score for trained models, and *last-query accuracy* for evaluations during training (Appendix A.4). The *acquisition step* of a hop length is the first evaluation during training at which its last-query accuracy reaches 0.8.

4 Results

Our starting point is the finding of Sesodia et al. (2026) that both families solve the task but organise the solution differently, with the looped model exposing the intermediate nodes one hop after another through depth (a staircase) and the untied model making them decodable only near the output, or not at all (Figure 3).

Training dynamics determine the staircase. That the untied model can solve the task at all shows that the task does not require a staircase, despite its strictly sequential nature. In this paper, we show that it is not a requirement of the *looped architecture* either. One of three looped seeds trained here solves $H = 8$ perfectly but with substantially diminished middle-hop selectivity (0.28–0.56 vs 0.80–0.97 in the other two). Another looped run, trained without the curriculum ramp (on the full hop range $H \in \{1, \ldots, 15\}$ from the start), solves $H = 8$ with no identifiable forward staircase, and the states it does expose near the input are consistent with inverse hops instead (Figure B.4).

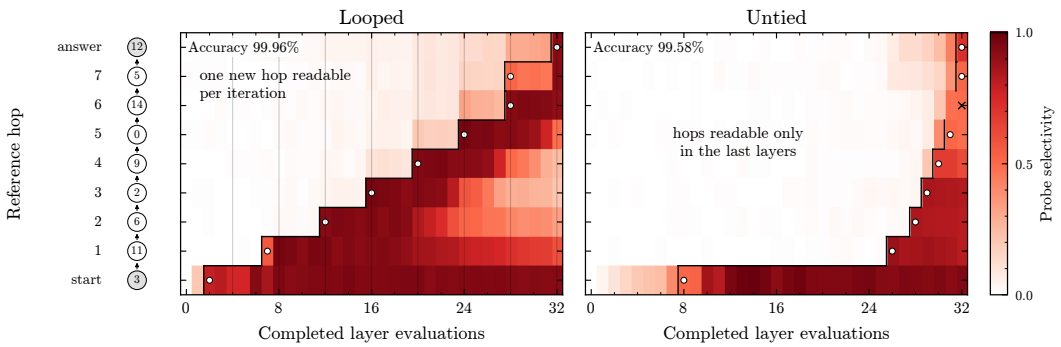


Figure 3: **Looped model forms a staircase.** Probe selectivity maps at $H = 8$ for the looped model (four-layer block, iterated eight times) and untied model (32 distinct layers). Both solve the pointer-chasing task (see far left-hand illustration), but display differing solution structures. White dots mark first decodability, the first layer at which a hop's selectivity reaches 0.5, and the black line traces them. Reproduced from Sesodia et al. (2026): matched-rate cohort ($\eta = 3 \times 10^{-4}$), seed 42; more details in Figure B.1.

Together, these results motivate the view that parameter sharing does not *dictate* the shape of a solution but *gently biases* the training process from which a solution emerges. If the layout is chosen by learning, the question is what exactly, during training under sharing, does the choosing. We identify the parameter update as the one quantity in training that differs between tied and untied models, since the looped model sums the contributions from its T uses.

4.1 Pooled gradients build the reusable hop operator

The key difference between the two families is that in the untied model each layer updates its own parameters, whereas in the looped model the T iterations share one set θ, which receives the sum of T hidden-state-specific contributions (Kaissis et al., 2026), resulting in the following gradient-descent update rules:

$$\begin{aligned} &\text{untied:} \quad \theta_t \leftarrow \theta_t - \eta\, g_t \qquad \text{for each } t, \\ &\text{looped:} \quad \theta \leftarrow \theta - \eta\, G, \qquad G = \nabla_\theta \mathcal{L} = \sum_{t=1}^{T} g_t. \end{aligned} \tag{3}$$

Here η is the learning rate and $g_t = \partial \mathcal{L} / \partial \theta_t$ is the gradient delivered to the parameters at their t-th use, which depends on the input $h^{(t-1)}$ the block received at that iteration.

The explicit summing of gradient contributions from different iterations favours changes that are useful at *every* iteration, as a direction that is shared by all T uses is added T times over, whereas a change that helps only one iteration is added once. We hypothesise that this gradient pooling is what is responsible for building the reusable one-hop operator, and not the architectural fact that the shared block is repeatedly applied.

Partial pooling. How much pooling is required to elicit the learning of this reusable one-hop operator? To answer this, we initialise T identical copies of the block, one per iteration, and update copy t with a partially pooled gradient $q_t = \alpha G + T(1 - \alpha)\, g_t$. At $\alpha = 1.00$ every copy receives $q_t = G$, which recovers the looped update; at $\alpha = 0.00$ each receives $q_t = T g_t$, which recovers the untied update (up to scale). Intermediate values of $\alpha \in \{0.25, 0.50, 0.75\}$ therefore investigate varying degrees of pooling.

We find that with full pooling ($\alpha = 1.00$) the model learns to solve every hop length up to $H = 8$ in all three seeds, and without pooling ($\alpha = 0.00$) it never does within the training budget (Figure 1). For intermediate values of α, more pooling makes the solution both more likely and earlier (Appendix B.2).

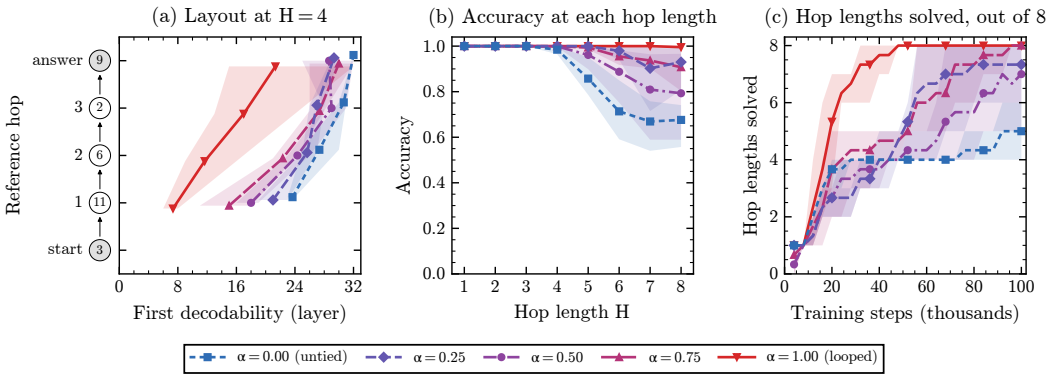


Figure 4: **Partial pooling.** We interpolate between full pooling (looped; $\alpha = 1$) and no pooled gradient contribution (untied; $\alpha = 0$) by updating copy t of our T identical blocks with $q_t = \alpha G + T(1-\alpha)\, g_t$. (a) First decodability of each reference hop at $H = 4$, i.e., the first layer at which its selectivity reaches 0.5, at the end of training. (b) Last-query accuracy at each hop length after training. (c) Hop lengths solved out of $H = 1, \ldots, 8$, i.e., the longest run of $H = 1, 2, \ldots$ at or above 0.8 last-query accuracy, over training.

Pooling also controls where in the network the solution is built (Figure 4). At $H = 4$, which every model solves, hop 1 first becomes decodable at layers 6–8 under full pooling and moves later, as α falls, to layers 23–24 without pooling. At $H = 8$, only full pooling builds the complete staircase. Every partially pooled model loses two to five hops (not decodable), and those it does decode from hop 3 onwards all lie in layers 25–32. This holds even for the three partially pooled models that solve $H = 8$ (accuracy 0.97–0.98), each of which still misses two or three of hops 5 to 7.

4.2 Neither a matched norm nor a matched average can replace the sum

Changing α may also change the size and noise of each copy's update. To isolate the sum itself, we return to the looped model ($\alpha = 1.00$), keep its forward pass unchanged, and substitute the full sum G (i) with: (ii) $G' = T\, g_T$, which keeps the final iteration's gradient; (iii) $G' = (\|G\|/\|g_T\|)\, g_T$, which is the same direction as (ii) just rescaled to the norm of the full sum; and (iv) $G' = T\, g_J$ with $J \sim \mathrm{Uniform}\{1, \ldots, T\}$, which draws one iteration randomly at each step and so keeps the expected update equal to G but never sums.

Figure 5(a) shows that only the full sum (i) solves $H = 8$ to completion, while none of the substituted G' (ii–iv) do. The final-iteration arms (ii) and (iii) stall at one or two hops in every seed, and the sampled arm (iv) fails even $H = 1$ in two seeds and plateaus at six hops in the third (Figure 5(b); Appendix B.2). Arm (iii) rules out magnitude, since it delivers the full norm and still fails, and arm (iv) rules out mean direction, since its expected update is G itself. Thus, within the training budget, the pooling of the gradient itself, i.e., the sum over uses, appears to be *the* critical ingredient that lets a reusable one-hop operator form, since building it up makes the solution appear (§4.1) and removing it prevents the solution from forming.

4.3 Evidence of the reusable one-hop operator

If pooling builds a reusable one-hop operator then, during training, intermediate-hop decodability and per-hop accuracy should improve in a *compositional burst* as the one-hop operator arrives and is composed across iterations, similar to the acquisition cascades of Kohli et al. (2026). An untied model has no such coupling, since each layer must learn its own transformation, so we expect it to acquire hop lengths one at a time. Both models here come from the shared-initialisation cohort (§3), so they start from the same function.

Compositional burst. The left and middle columns of Figure 6 indicate at which point *during training* intermediate hops first become decodable, and where in the network this happens. The looped model (left column) decodes only hop 1 for the first 44k steps, and then every remaining hop

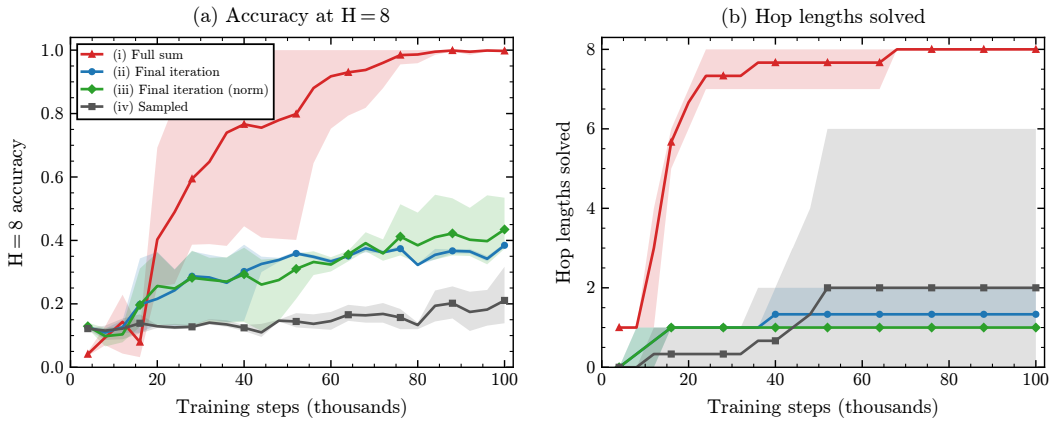


Figure 5: **Gradient routing.** The forward computation stays tied while the gradient applied to the shared block is (i) the full sum G, which is ordinary looped training; (ii) the final iteration's contribution alone, scaled by T; (iii) the same final-iteration direction rescaled to the norm of the full sum; or (iv) the contribution of one iteration drawn uniformly at random each step, scaled by T. (a) Last-query accuracy at $H = 8$, evaluated every 4k training steps. (b) Hop lengths solved: the longest run of hop lengths from $H = 1$ up to $H = 8$ with last-query accuracy of at least 0.8 (Appendix A.4).

becomes decodable within the following 8k steps. The untied model (middle column) acquires hop lengths one at a time at $H = 4$ and, at $H = 8$, hops 3 to 7 exhibit a gulf of no decodability before the final answer appears at the final layer. This burst does not exist solely in decodability-space, but also in accuracy-space. Figure 6(e) (top right) shows that the looped model acquires $H = 3$ to 8 at the same evaluation, whereas the untied model acquires hop lengths one at a time, reaching $H = 8$ only at 76k steps in the seed shown, and stalls at $H = 6$ after a partial burst in the other seed. Figure 6(f) (bottom right) shows the final answer accuracy. A second seed is shown in Appendix Figure B.5.

Within-block analysis. To verify that each iteration performs the same operation, we consider each of the 32 effective layers of the looped model modulo $L = 4$ and ask at which point within the block the hop result becomes decodable. In two of three seeds, the hop state becomes decodable only in the fourth and final block layer. In the clearest seed, skipping that layer at any one of the eight iterations collapses $H = 8$ accuracy to 0.2–0.36, while skipping an interior layer costs little. The untied models show no preferred position; the full analysis is in Appendix B.3.

4.4 Pooling helps most when the pooled iterations span the traversal

Pooling could build the hop operator by combining learning signal from every stage of the computation, or simply because the shared parameters are trained more. To distinguish the two, we train models with *two* shared blocks that have distinct parameter sets, keep the training per parameter fixed, and vary only where in the computation each block appears.

Denoting the two parameter groups A and B, *contiguous* sharing (AAAABBBB) runs A at iterations 1 to 4 and B at iterations 5 to 8, so each group meets one stretch of the traversal; *cyclic* sharing (ABABABAB) alternates them, so each meets states from across the whole traversal. The same pair was compared by Takase & Kiyono (2023) and in Mixture-of-Recursions (Bae et al., 2025), where cyclic reuse gave lower pretraining loss than sequential reuse at the same parameter and reuse counts, but no reason was given. The looped (AAAAAAAA) and untied (ABCDEFGH) models of the same cohort serve as reference points (Appendix B.4). If sharing acts only through the amount of training per parameter, cyclic and contiguous should behave similarly. If it acts by making one set of weights serve every stage of the computation, cyclic should behave like the looped model and contiguous like the untied one.

Cyclic sharing outperforms contiguous sharing. Figure 7(a,b) shows that cyclic sharing solves $H = 8$ before contiguous sharing in all three seeds. It also places the staircase earlier in the network

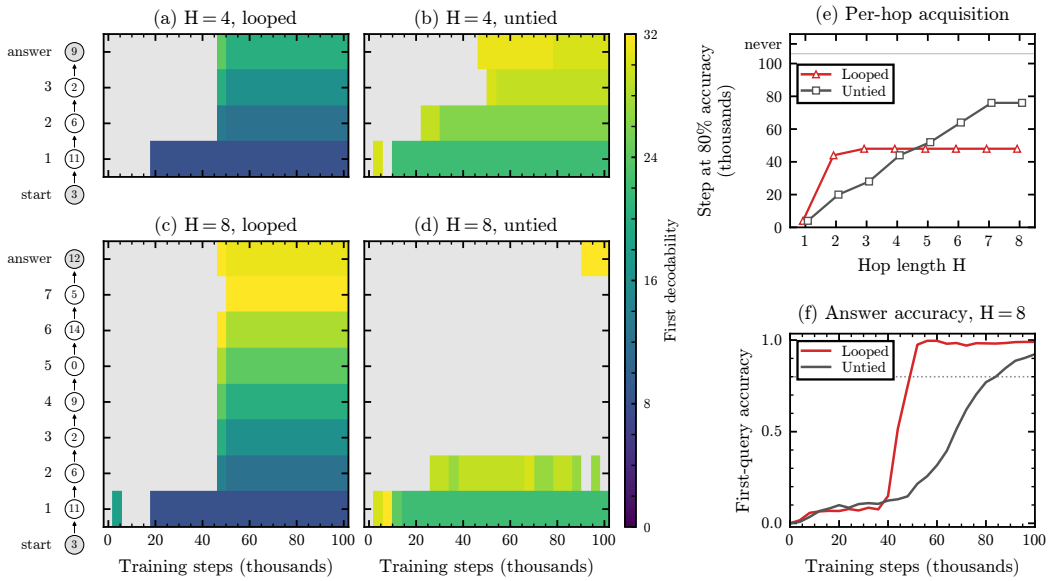


Figure 6: **Staircase formation during training.** (a–d) First decodability (first layer at which each intermediate hop's selectivity reaches 0.5) every 4k training steps; grey marks a hop with no crossing at that snapshot. (e) Per-hop acquisition: the training step at which last-query accuracy first reaches 80% for each hop length. (f) Complete first-query accuracy at $H = 8$; the dotted line marks 0.8. More details in Appendix Figure B.5.

(Figure 7(c)): hop 1 becomes decodable at layers 11 and 12 under cyclic sharing and at layers 20 and 17 under contiguous sharing, which is where the untied model places it, despite sharing four to one. In seeds 42 and 43 every pattern keeps a complete $H = 4$ staircase (Table B.6), but at $H = 8$ only the looped model keeps all eight hops decodable, while both two-block patterns and the untied model lose middle hops (Figure 7(c)). That the two diverge at all, and that cyclic is the stronger, rules out the interpretation that shared parameters are simply trained more. A group therefore learns fastest with gradient from across the computation, consistent with the failure of the single-iteration arms in §4.2. One caveat is that A's output passes through B before A recurs, so the cyclic result is also consistent with a two-block unit AB being reused rather than one block composing with itself.

5 Discussion and Limitations

Expressivity. It is worth ruling out some simpler explanations for the observed staircase, before committing to our account. At matched depth, weight tying can only *remove* capacity, so our looped models are actually less expressive than their untied counterparts (3.2M against 25.7M). This reduced capacity does suggest a subtler reading: perhaps the per-iteration solution is simply the only one a looped model *can* learn. The gradient-routing experiments (§4.2) test this directly, since they hold the forward pass (and hence the capacity) fixed and change only the magnitude and aggregation of the gradient signal. Every altered variant fails to solve $H = 8$, including one that receives the full update norm, which also leaves little room for an explanation based on more gradient per parameter. The partial-pooling models make the same point from the other side: each has the capacity of the untied model, yet only full pooling builds the complete staircase (§4.1).

Learning-rate sensitivity. Untied models are more sensitive to the learning rate than looped models (Sesodia et al., 2026; Table A.4). The looped model solves at $\eta = 10^{-3}$ and 3×10^{-4} but not at 10^{-4}, whereas the untied model solves reliably only at 3×10^{-4}. Our intuition is that, because one shared update is read back at every iteration (Li et al., 2026), gradient pooling confines the search to a region in which the reusable operator is easier to reach. The untied model, in contrast, must find mutually compatible transformations in 32 independent layers. This is consistent with the loss-landscape bias conjectured for recursion by Gong et al. (2025).

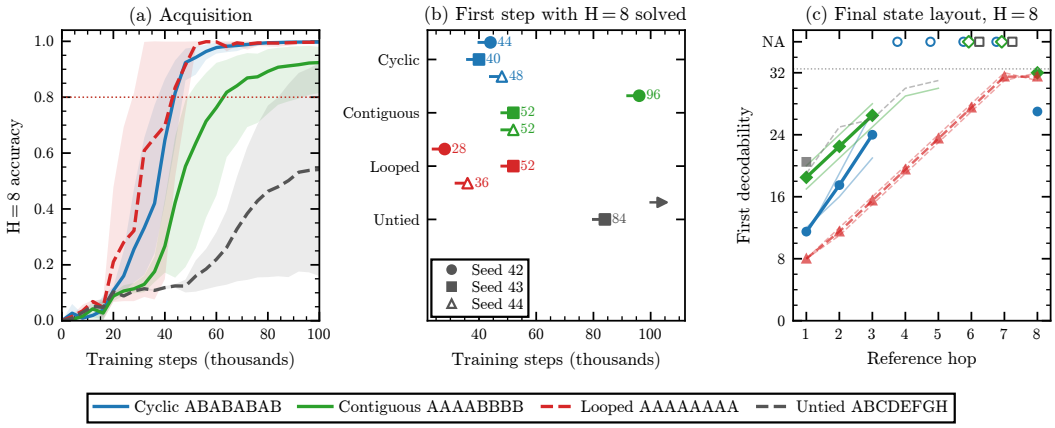


Figure 7: **Parameter-sharing patterns.** (a) $H = 8$ evaluation accuracy over training; lines are the mean of three seeds (two for untied), shaded area indicates the range; the dotted line marks 0.8. (b) During training, first evaluation at which (a) reaches 0.8, one point per seed; an arrow at the right edge marks not reached by 100k steps. (c) First decodability on the trained model for $H = 8$; thin lines for seeds 42 and 43 and their mean with markers; hollow markers on the NA row are hops decodable in neither seed.

Probes. We test our use of probes with causal interventions. Specifically, transplanting a hop's state from a donor prompt redirects the remaining traversal in about half of trials, while control edits never do, although in some models transplanting a readable state leaves the answer unchanged (Table B.4). Linear probes can also miss information that is encoded non-linearly, and readouts of looped states may depend on the decoder (Lu et al., 2025; Sharma & Vu, 2026).

Scope. We chose DEPO because it is *the* minimal sequential task with uniquely defined intermediate states, and the building block from which iterative algorithms such as graph traversal are composed. All results come from this one task, with models of 3M to 26M parameters and typically three seeds per condition. We tried hardening the task to $N = 32$ nodes, but found neither family solves all hop lengths $H \leq 15$, although we did not recalibrate optimisation from the $N = 16$ recipe. The looped model still gets further, solving the first six hop lengths in two of three seeds, while the untied model solves only the first. We also tried a second task, a register machine in which the model reports the final value of one of four registers after a straight-line program of up to 15 arithmetic and conditional-move instructions. However, it did not test the staircase as intended, since most instructions do not affect the queried register. E.g., on average, the answer to a 15-instruction program depends on only 6.5 of them (Appendix C). Probing the query position shows that only the final answer is linearly readable, so there is no DEPO-like staircase, which is expected when there are almost no intermediate states to lay out. Testing the account on a second task with long serial dependencies, such as graph search, and applying partial pooling to pretrained looped language models are natural next steps.

6 Conclusion

Looped transformers solve sequential pointer chasing along a staircase of intermediate states (Sesodia et al., 2026), which we find is decided by training, since neither the task nor the architecture forces it (§4). The selecting signal is the pooled gradient from multiple uses of the shared block, which favours the construction of a reusable one-hop operator (§4.1), and this works best when the block receives gradient from every stage of the traversal (§4.4). Once formed, this operator is self-composed to fully solve the task, leading to the gradual staircase-shaped solution (§4.3). The aforementioned iterative inductive bias of looped transformers is therefore a property of learning under shared parameters. This means that, as looped models scale beyond tens of billions of parameters, the bias can be controlled through careful design of the backward pass.

AI USE STATEMENT

We used AI to design and execute implementation code, including the procedural task generator, and to make results tables and figures. We also used AI to find related work, draft parts of the paper, polish manuscript prose, and improve readability. We used Anthropic's Claude Opus 5.0, 5.5 & Fable 5.1, as well as OpenAI's GPT-6 Astra. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of AI.

ETHICS STATEMENT

All experiments use procedurally generated graph tasks, so no personal data, human subjects or scraped text are involved. The models are small and trained only on this synthetic task, and we see no direct route to misuse. Our results describe how these models compute in a controlled setting and make no claim about how reliably deployed language models reason.

REPRODUCIBILITY STATEMENT

Appendix A gives the full implementation details: task generation, architectures, training recipe and seeds, metrics, and every probe, intervention and training protocol. The code for every run, figure and table is public under the MIT license (Appendix A.8), which also states the compute required.

REFERENCES

Zeyuan Allen-Zhu. Physics of language models: Part 4.1, architecture design and the magic of canon layers. In *NeurIPS*, 2025.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *NeurIPS*, 2025.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *NeurIPS*, 2019.

Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to ponder. *arXiv preprint arXiv:2107.05407*, 2021.

Hugh Blayney, Álvaro Arroyo, Johan Obando-Ceron, Pablo Samuel Castro, Aaron Courville, Michael M. Bronstein, and Xiaowen Dong. A mechanistic analysis of looped reasoning language models. *arXiv preprint arXiv:2604.11791*, 2026.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *ICLR*, 2019.

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped transformers for length generalization. In *ICLR*, 2025.

Zitian Gao, Lynx Chen, Yihao Xiao, He Xing, Ran Tao, Haoming Luo, Joey Zhou, and Bryan Dai. Universal reasoning model. *arXiv preprint arXiv:2512.14693*, 2025.

Zitian Gao, Yilong Chen, Yihao Xiao, Xinyu Yang, Ran Tao, Joey Zhou, and Bryan Dai. Loop the Loopies! *arXiv preprint arXiv:2607.16051*, 2026.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *NeurIPS*, 2025.

Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *ICML*, 2023.

Zixuan Gong, Yong Liu, and Jiaye Teng. What makes looped transformers perform better than non-recursive ones. *arXiv preprint arXiv:2510.10089*, 2025. URL https://arxiv.org/abs/2510.10089.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Sang-Il Han. Hierarchical vs. flat iteration in shared-weight transformers. *arXiv preprint arXiv:2604.14442*, 2026.

Jaemu Heo, Eldor Fozilov, Hyunmin Song, and Taehwan Kim. RingFormer: Rethinking recurrent transformer with adaptive level signals. In *Findings of EMNLP*, pp. 21675–21686, 2025.

John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *EMNLP-IJCNLP*, pp. 2733–2743, 2019. doi: 10.18653/v1/D19-1275. URL https://aclanthology.org/D19-1275/.

Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. Loopformer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *ICLR*, 2026.

Georgios Kaissis, David Mildenberger, Juan Felipe Gomez, Martin J. Menten, and Eleni Triantafillou. Step-resolved data attribution for looped transformers. In *ICML*, 2026.

Jonas Knupp, Jan Hendrik Metzen, Jeremias Bohn, Georg Groh, and Kristian Kersting. Depth-recurrent attention mixtures: Giving latent reasoning the attention it deserves. *arXiv preprint arXiv:2601.21582*, 2026.

Harsh Kohli, Srinivasan Parthasarathy, Huan Sun, and Yuekun Yao. Loop, think, & generalize: Implicit reasoning in recurrent-depth transformers. In *COLM*, 2026.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *ICLR*, 2020.

Chen-Yu Lee, Saining Xie, Patrick Gallagher, Zhengyou Zhang, and Zhuowen Tu. Deeply-supervised nets. In *AISTATS*, 2015.

Shuzhen Li, Yifan Zhang, Jiacheng Guo, Quanquan Gu, and Mengdi Wang. DeepLoop: Depth scaling for looped transformers. *arXiv preprint arXiv:2607.13491*, 2026. URL https://arxiv.org/abs/2607.13491.

Wenquan Lu, Yuechuan Yang, Kyle Lee, Yanshu Li, and Enqi Liu. Latent chain-of-thought? decoding the depth-recurrent transformer. *arXiv preprint arXiv:2507.02199*, 2025.

Sean McLeish, Ang Li, John Kirchenbauer, Dayal Singh Kalra, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Jonas Geiping, Tom Goldstein, and Micah Goldblum. Teaching pretrained language models to think deeper with retrofitted recurrence. In *COLM*, 2026.

William Merrill and Ashish Sabharwal. A little depth goes a long way: The expressive power of log-depth transformers. In *NeurIPS*, 2025a.

William Merrill and Ashish Sabharwal. Exact expressive power of transformers with padding. In *NeurIPS*, 2025b.

Michael C. Mozer, Shoaib Ahmed Siddiqui, and Rosanne Liu. The topological trouble with transformers. *arXiv preprint arXiv:2604.17121*, 2026.

Francesco Pappone, Donato Crisostomi, and Emanuele Rodolà. Two-scale latent dynamics for recurrent-depth transformers. *arXiv preprint arXiv:2509.23314*, 2025.

Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling laws for stable looped language models. In *COLM*, 2026.

Nikunj Saunshi, Stefani Karp, Shankar Krishnan, Sobhan Miryoosefi, Sashank J. Reddi, and Sanjiv Kumar. On the inductive bias of stacking towards improving reasoning. In *NeurIPS*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/837bc5db12f3d394d220815a7687340c-Abstract-Conference.html.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *ICLR*, 2025.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. How much is one recurrence worth? Iso-depth scaling laws for looped language models. *arXiv preprint arXiv:2604.21106*, 2026.

Magnus Sesodia, Philip Torr, and Christian Schroeder de Witt. Inside the loop: A mechanistic study of weight-tied transformers on depth-bound algorithmic tasks. In *NeurIPS*, 2026. To appear.

Rituraj Sharma and Tu Vu. Dense supervision is not enough: The readout blind spot in looped language models. *arXiv preprint arXiv:2606.24898*, 2026.

Sho Takase and Shun Kiyono. Lessons on parameter sharing across layers in transformers. In *Proceedings of the Fourth Workshop on Simple and Efficient Natural Language Processing (SustaiNLP)*, pp. 78–90, 2023.

Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.

Shaowen Wang, Bingrui Li, Ge Zhang, Wenhao Huang, Shen Yan, and Jian Li. On the residual scaling of looped transformers: Stability and transferability. *arXiv preprint arXiv:2606.18524*, 2026a.

Shaowen Wang, Ge Zhang, Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, and Jian Li. SMELT: Scaling laws for compute-matched MoE looped transformers. *arXiv preprint arXiv:2609.01343*, 2026b.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel loop transformer for efficient test-time computation scaling. *arXiv preprint arXiv:2510.24824*, 2025.

Kevin Xu and Issei Sato. On expressive power of looped transformers: Theoretical analysis and enhancement via timestep encoding. In *ICML*, 2025.

Kevin Xu and Issei Sato. A formal comparison between chain of thought and latent thought. In *ICML*, 2026.

Jian Yang, Wei Zhang, Shawn Guo, Zhengmao Ye, Lin Jing, Shark Liu, Yizhi Li, Jiajun Wu, Cening Liu, X. Ma, Yuyang Song, Siwei Wu, Yuwen Li, L. Liao, T. Zheng, Ziling Huang, Zelong Huang, Che Liu, Yan Xing, Renyuan Li, Qingsong Cai, Hanxu Yan, Siyue Wang, Shikai Li, Jason Klein Liu, An Huang, Yongsheng Kang, Jinxing Zhang, Chuan Hao, Haowen Wang, Weicheng Gu, Ran Tao, Mingjie Tang, Peihao Wu, Jianzhou Wang, Xianglong Liu, Weifeng Lv, and Bryan Dai. IQuest-Coder-V1 technical report. *arXiv preprint arXiv:2603.16733*, 2026.

Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. In *ICLR*, 2024.

Chengting Yu, Xiaobo Shu, Yadao Wang, Yizhen Zhang, Haoyi Wu, You Wu, Rujiao Long, Ziheng Chen, Yuchi Xu, Wenbo Su, and Bo Zheng. Spiralformer: Looped transformers can learn hierarchical dependencies via multi-resolution recursion. *arXiv preprint arXiv:2602.11698*, 2026.

Tong Zhang, Junhao Hu, Yun Peng, and Tao Xie. When does recurrence become an algorithm? Convergence selection in weight-tied looped transformers. *arXiv preprint arXiv:2607.20594*, 2026.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025.

A SETUP

A.1 TASK AND PROMPT FORMAT

An example begins with a uniformly sampled ordering $(v_0, \ldots, v_{N-1})$ of the node labels, whose edges $v_i \mapsto v_{(i+1) \bmod N}$ form one directed N-cycle; the displayed edge order is shuffled independently. Each prompt then asks $M = 5$ queries over the same edge list, all at the same hop count H, with start nodes drawn independently with replacement (Figure A.1).

Training is effectively online. There are $15! \approx 1.3 \times 10^{12}$ cycles on 16 nodes, and a 100k-step run at 120 prompts per batch draws $n = 1.2 \times 10^7$ prompts, so the expected number of prompt pairs that share a cycle is $n(n-1)/(2 \cdot 15!) \approx 55$, about one prompt in 10^5. Each cycle has $16! \approx 2.1 \times 10^{13}$ displayed edge orders, so an exact repeat of an edge list is not expected in any run, and an evaluation prompt's cycle has appeared in training with probability below 10^{-5}.

We tokenize with `hf-internal-testing/llama-tokenizer` and remap the observed token IDs to a compact vocabulary of 18 tokens: the ten digits, `hop`, `>`, `,`, `|`, `?`, a bare whitespace token, and the beginning- and end-of-sequence tokens. Node labels are decimal strings split into digits, so nodes 10 to 15 take two tokens and the 16 labels need only the ten digits. The map is rebuilt by generating 50 prompts at each training difficulty with Python random seed 999, collecting their token IDs together with the special tokens, and sorting the original IDs; a missing padding token is set to the end-of-sequence token, giving compact padding ID 1. The context is 200 tokens, padded to that width, with the attention mask passed to every layer.

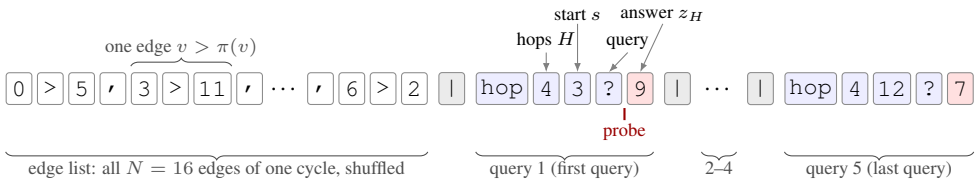


Figure A.1: **Prompt format.** One prompt with the example of Figure 2 ($s = 3$, $H = 4$, answer 9). The edge list is followed by $M = 5$ queries at the same H, separated by `|`, each with its own start node; queries 2 to 4 and most edges are elided. Nodes are decimal strings, so 10 to 15 are two digit tokens. First-query accuracy scores query 1; last-query accuracy scores query 5, with the four earlier answers in context. Probes read the whitespace token between `?` and the answer.

A.2 MODELS

Both families are causal decoder-only transformers with pre-normalized residual attention and feed-forward sublayers (Table A.1). Attention has eight heads of dimension 32, bias-free projections and rotary position embeddings (RoPE) with base 10,000. Each feed-forward sublayer is a SwiGLU whose hidden width is $\lfloor 2(4d)/3 \rfloor$ rounded up to a multiple of 64. All normalizations are RMSNorm with $\epsilon = 10^{-5}$, and the readout is a bias-free linear map to the vocabulary, untied from the embedding. There is no dropout, input re-injection, prelude or coda. The iteration embeddings τ_t (Equation 1) are initialized from $\mathcal{N}(\mu{=}0, \sigma{=}0.02)$. In the looped implementation the attention output projection and the last SwiGLU projection of each layer are initialized to zero; the untied model keeps the PyTorch default initialization. Both families are trained with full backpropagation through all $LT = 32$ layer evaluations, with no truncation, so the shared block receives the sum of its T per-iteration gradients.

Table A.1: Architectures of the matched comparison. A layer evaluation is one attention and one feed-forward sublayer. Parameter counts include embeddings, readout, normalization and iteration embeddings. The 2×16 and 8×4 geometries change only L and T.

Property	Looped 4×8	Untied 32-layer
Residual width d / attention heads	256 / 8	256 / 8
SwiGLU hidden width	704	704
Distinct transformer layers L	4	32
Iterations T	8	1
Layer evaluations LT	32	32
Iteration embeddings τ_t	$T \times d = 8 \times 256$	none
Trainable parameters	3,224,832	25,715,968

A.3 Training recipe and cohorts

At optimizer step t the largest active hop count is

$$H_{\max}(t) = \begin{cases} 1, & t \le 7000, \\ \min\big(15, \lfloor 1 + 14(t-7000)/23000 + \frac{1}{2} \rfloor\big), & 7000 < t < 30000, \\ 15, & t \ge 30000, \end{cases} \tag{4}$$

and each batch of 120 prompts is divided as evenly as possible over $H = 1, \ldots, H_{\max}(t)$, remainders going to the lowest difficulties, then shuffled. Every example runs all 32 layer evaluations whatever its H. The loss covers every non-padding target of the sequence (edges, query syntax and answers). With scheduler index u, the learning-rate multiplier is $u/1000$ for $u < 1000$ and $0.1 + 0.45[1 + \cos(\pi(u - 1000)/99000)]$ afterwards, and AdamW receives all parameters in one group (Table A.2). Python, NumPy, PyTorch and CUDA are seeded with the training seed, and batch generation uses a separate `Random(seed)` instance.

Table A.2: Training settings shared by every cohort, taken from the saved run arguments. The training-intervention and shared-initialisation cohorts keep every 4,000-step snapshot.

Setting	Value
Budget	100,000 optimizer steps
Batch / context	120 prompts / 200 tokens
Task	$N = 16$, $M = 5$, $H = 1, \ldots, 15$
Curriculum	7,000-step floor, 23,000-step ramp
Optimizer	AdamW, $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.01
Peak learning rate	3×10^{-4} (10^{-3} in the learning-rate cohort)
Warmup / decay	1,000 steps linear; cosine to 10% of peak
Gradient clipping	global ℓ_2 norm 1.0
Precision	float32 (routing and pooling reduce gradient norms in float64)
Evaluation / checkpoint interval	4,000 / 20,000 steps

The paper uses four cohorts trained under this recipe (Table A.3). In the shared-initialisation cohort one looped 4×8 initialization is drawn per seed and its block is copied into every parameter group of every arm, embeddings, readout and iteration embeddings included, so all arms of a seed compute the same function at step 0 (the untied arm starts from eight identical blocks) and differ only in how their parameters are tied during training. The matched pair was also trained at two further peak learning rates (Table A.4); at 10^{-3} no untied depth from 2 to 32 layers succeeds in any of five seeds.

Table A.3: Training cohorts.

Cohort	Runs	Used in
Matched-rate	Looped 4×8 and untied 32-layer, seeds 42–44	Figures 3, B.1; Appendices B.1, B.3
Learning rate 10^{-3}	Looped 2×16; 4×8 without iteration embeddings, seeds 42–45; register machine	Figure B.1; Appendices B.1, C
Training interventions	Gradient routing (4 rules) and partial pooling (5 values of α), seeds 42–44	Figures 1, 4, 5; Appendix B.2
Shared initialisation	Looped, cyclic, contiguous (seeds 42–44) and untied (42, 43) partitions; 8×4 (42, 43); looped without curriculum (42)	Figures 6, 7; Appendices B.1, B.3, B.4

Table A.4: Seeds whose solved prefix reaches 15 within 100k steps, out of seeds trained, for the matched pair at three peak learning rates.

Family	10^{-3}	3×10^{-4}	10^{-4}
Looped 4×8	5/5	3/3	0/3
Untied 32-layer	0/5	3/3	1/3

A.4 Metrics

Table A.5 lists the two answer scores. Both require the complete answer: every digit and the following delimiter (or, for query 5, every digit) must equal the argmax under the gold prefix, so the leading digit shared by nodes 1 and 10 to 15 earns no partial credit and chance is $1/16$. The last-query evaluator runs every 4k steps on $\lfloor 8000/15 \rfloor = 533$ fresh prompts per difficulty (generator seed $99999 + t + H$); its binomial standard error is at most 0.022 per point.

Table A.5: The two answer scores. Prompt counts are per condition and difficulty.

Score	Definition	Prompts	Used in
First-query	Complete answer of query 1, no earlier answer in context	5,000 (checkpoints); 1,000 (snapshots)	Figures 3, B.1, 6(f), 7; Appendix B.1
Last-query	Answer of query 5 under teacher forcing, four gold answers in context	533; 500 in the knockout	Figures 1, 4, 5, 6(e), B.6(b); Tables A.4, B.2, B.5

Last-query accuracy admits a copy shortcut. Query k repeats an earlier start with probability $1 - (15/16)^{k-1}$, 0.228 for the fifth query, in which case its answer is already in context, so a model that copies such answers and otherwise guesses scores about 0.28 on last-query accuracy and 0.06 on first-query accuracy at any H. The threshold 0.8 lies above both; sub-threshold last-query values, such as the altered routing arms near 0.4 in Figure 5(a), include copying.

The *solved prefix* of a run, used in the appendix tables, is the largest $h \leq 15$ such that last-query accuracy is at least 0.8 for every $H \leq h$, or zero if $H = 1$ fails. A run that never crosses within 100k steps is right censored and plotted as not reached, and shaded bands show the range over seeds.

A.5 Linear probes

For each trained checkpoint and $H \in \{4, 8\}$ we generate 5,000 prompts with evaluation seed 12345 and read all five query states, 25,000 rows; examples, targets and split are identical across architectures and seeds at a given H. The read position is the whitespace token after `?`, immediately before the first answer digit (Figure A.1). We keep the embedding state and the residual state after each of the 32 layer evaluations, before the final normalization, giving 33 levels for every model; in the looped model the state after iteration t is read before τ_{t+1} is added. A NumPy generator with seed 0 assigns 4,000 whole prompts to fitting and 1,000 to evaluation, so no prompt contributes to both. Each probe is fitted on the unstandardized float32 states for 30 epochs with AdamW (learning rate

10^{-3}, weight decay 10^{-4}, batch 256), without early stopping or schedule; the control probe uses the same recipe with labels permuted within the fitting split and is scored against the original held-out labels. Onsets use threshold 0.5, with 0.4, 0.6 and 0.7 as sensitivities; a hop that never crosses stays missing, and adjacent onsets may be equal or inverted. No probe of a final checkpoint is fitted unless first-answer-digit accuracy is at least 0.8 on 500 prompts. The formation, sharing and readability analyses probe the snapshots saved every 4k steps (26 from step 0 to 100k) with the same recipe on 1,000 prompts per H, and score first-query accuracy on the same prompts; the accuracy gate does not apply to them, since early snapshots do not yet solve the task.

A.6 Causal interventions

State transfer. State transfer runs on the matched-rate checkpoints at $H = 4$. For hops $j \in \{1, 2, 3\}$ we edit at the probe onset of hop j and at the two neighbouring layers, keeping depths below 32 so that computation remains after the edit; the selection uses no intervention outcome. Each hop has 128 donor–receiver trials (generator seed $202609150 + 10000j$), reused at its three depths. Receiver and donor have independent cycles π_R, π_D and the same first-query start s; the donor's hop-j node is $z_D = \pi_D^j(s)$, and the composition target $y = \pi_R^{H-j}(z_D)$ is required to differ from both original answers, so copying cannot score. Intervention probes are refitted on the first queries of the 4,000 fitting prompts (seed $202609150 + 100\ell + j$). With W the fitted probe matrix, its rows centred over classes, U spans the singular directions above 10^{-5} times the largest singular value (rank at most 15), and the node-subspace edit at depth ℓ is $\widetilde{h}_R = h_R + UU^\top(h_D - h_R)$ at the first-query read position (Table A.6). Future tokens are hidden while every observed token, position and padding mask is kept; the model then greedily decodes up to five tokens, stopping at a delimiter, with the edited state reinserted on every forward pass. A trial counts only if the output is the complete target node followed by `|`. Identity edits reproduce the unpatched logits and answers exactly.

Table A.6: State-transfer conditions, all evaluated on the same 128 trials per hop and depth.

Condition	Construction
Unpatched / identity	Ordinary forward pass / rewrite the original state.
Whole state	Replace h_R by h_D.
Node subspace	Add $UU^\top(h_D - h_R)$; the primary edit.
Same-node alternative	A second donor context with the same start and hop-j node, projected as above.
Shuffled node	A donor carrying a different eligible node, scored on the primary target and on its own composition target.
Orthogonal	The donor difference projected onto a fixed random subspace orthogonal to U of equal rank (seed $202609150 + 1000\ell + j$), rescaled per trial to the node edit's norm.

Layer knockout. Knockout removes one layer evaluation, or a window of eight, in the unrolled 32-layer computation, so a looped layer is removed at one iteration only. *Skipping* returns the layer's input unchanged; *mean ablation* replaces the layer's contribution (output minus input) by its mean over a fixed reference batch of 128 $H = 8$ prompts (seed 0). Accuracy is last-query accuracy on 500 prompts per hop length with generator seed 99999.

A.7 Training interventions

Gradient routing. At every step the shared block is copied into eight equal-valued parameter leaves, one per iteration, whose gradients g_t are computed with every hidden-state path intact; only their aggregation into the block update changes. The iteration J of rule (iv) is drawn from a dedicated generator seeded with the training seed plus 10,000. Embeddings, iteration embeddings and readout keep their ordinary gradients; global clipping and one AdamW step follow, and the updated block is recopied into the leaves at the next step. Norm matching applies to the raw block gradient; because clipping acts on all parameters together and AdamW normalizes per coordinate, equal routed norms do not imply equal updates, so each arm measures the total effect of its rule including these optimizer consequences. Over the three seeds, during the first 24k steps the global clip is active on 3.2 to 3.7% of steps for rules (i) and (iv), 18 to 29% for (ii) and 29 to 78% for (iii). Snapshots every 4k steps

store model, optimizer, scheduler and every generator state, and a resume test reproduced sampled iterations, batches and parameters exactly.

Partial pooling. The eight block copies share one clipping coefficient, set by $N^2 = \|g_{\text{other}}\|^2 + \frac{1}{8}\sum_t \|q_t\|^2$ over the non-block gradients g_{other} and the pooled updates q_t. At $\alpha = 1$ the copies and their optimizer moments stay equal, so the run is looped training.

Sharing partitions. The looped, two-group and untied partitions contain 3,224,832, 6,438,144 and 25,718,016 parameters; the untied count exceeds Table A.1 by the iteration embeddings, which every partition keeps. The looped run without the curriculum samples $H = 1, \ldots, 15$ uniformly from step 0.

A.8 Code and compute

The code for every run, figure and table is released under the MIT license at `https://anonymous.4open.science/r/looped-gradient-pooling-F319`. The reported results took about 1,700 GPU-hours, mostly on NVIDIA A40 GPUs, and one 100k-step training run takes about 17 hours on an A40.

B Supporting results

B.1 Solution organisation (§4)

Probe maps. Figure B.1 shows the probe maps of the matched-rate cohort for seed 42, next to a looped 2×16 model trained at $\eta = 10^{-3}$, the learning rate of Sesodia et al. (2026). Table B.1 lists the accuracy, onsets and threshold sensitivity of all six matched-rate checkpoints, and Figure B.2 shows their selectivity maps. Figure B.3 shows the same maps for an eight-layer block applied four times.

Table B.1: Looped 4×8 and untied 32-layer models of the matched-rate cohort (the checkpoints of Figure B.2): first-query accuracy (%, 5,000 prompts), onsets at selectivity 0.5 in layer evaluations (a dash marks no crossing), and the number of hops without a crossing at thresholds 0.4, 0.6 and 0.7.

	Seed	H	Acc.	Onsets for hops $1, \ldots, H$	0.4	0.6	0.7
Looped	42	4	100.00	7, 12, 16, 20	0	0	0
	43	4	100.00	8, 13, 20, 24	0	0	0
	44	4	100.00	7, 10, 12, 16	0	0	2
Untied	42	4	99.98	26, 28, 29, 30	0	0	0
	43	4	100.00	17, 25, 29, 30	0	0	0
	44	4	98.16	25, 28, 32, 31	0	2	4
Looped	42	8	99.96	7, 12, 16, 20, 24, 28, 28, 32	0	0	0
	43	8	99.52	8, 12, 16, 20, 24, 28, 28, 27	0	0	0
	44	8	100.00	7, 10, 14, 15, 19, 22, 24, 28	0	5	6
Untied	42	8	99.58	26, 28, 29, 30, 31, –, 32, 32	0	3	4
	43	8	89.88	17, 29, –, –, –, –, –, 32	2	6	7
	44	8	94.64	25, –, –, –, –, –, –, 31	4	7	8

Some finer details. First, the looped model retains each intermediate state for roughly two iterations after it appears, most visible as the hollow region inside the $H{=}8$ staircase, and less so for $H = 4$. This is consistent with a reusable hop operation whose inputs need only persist until they have been consumed. Second, the looped model spends its first iteration "warming up", as hop 1 becomes decodable only during the second iteration. We conjecture that the first iteration must be spent casting the input into a form the hop operator can meaningfully act upon, which means that for $H{=}8$, where the iteration count equals the hop count, a double-hop is required to compensate for the slow start. Third, although progress is not decodable in the early and middle layers of the untied model, these layers are not idle. Skipping layers 1–8 reduces $H{=}8$ accuracy from 1.000 to 0.082, and removing any other eight-layer window is similarly damaging (Table B.2).

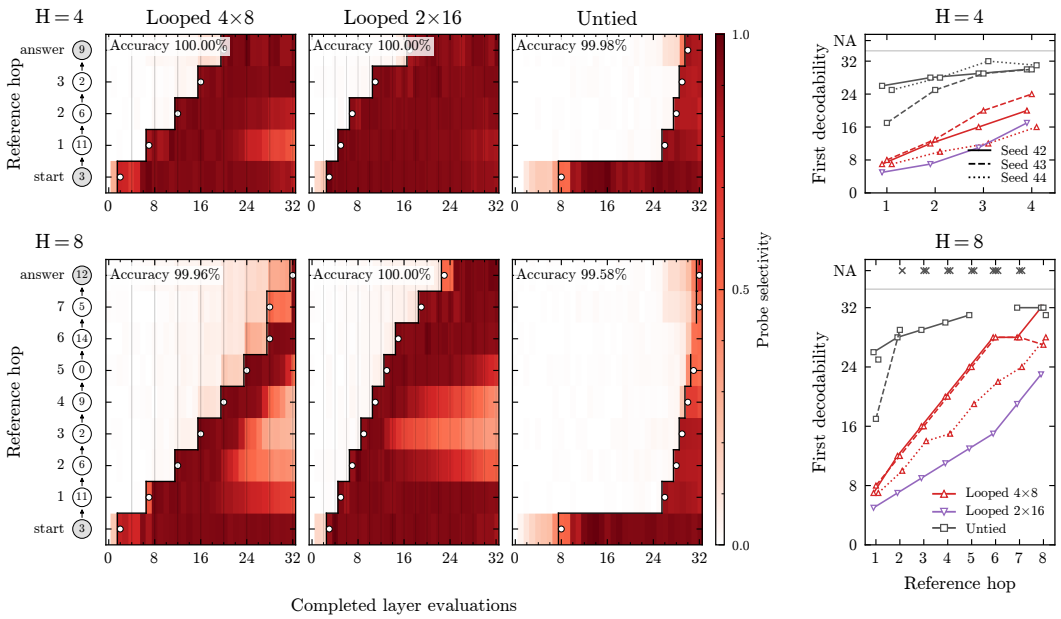


Figure B.1: **Solution staircases: looped vs. untied.** Left three columns: probe selectivity maps for a looped model with L=4 layers applied T=8 times (left), L=2 layers applied T=16 times (middle), and an untied L=32 layer model (right) at H=4 (top) and H=8 (bottom). The 4×8 and untied models use learning rate 3×10^{-4}, whereas the 2×16 model uses 10^{-3}; the training recipe is otherwise identical (Table A.3). All maps show seed 42. The node chain at the far left illustrates the meaning of the y-axis with a single example trace; the maps themselves are fitted on 4,000 prompts and evaluated over 1,000 held-out prompts. White dots and black lines denote first decodability, i.e., the first layer at which a reference hop's selectivity reaches 0.5. Faint grey lines mark block boundaries. Right column: first decodability for all three models, with seeds 42, 43 and 44 of the 4×8 and untied models drawn solid, dashed and dotted, and the single 2×16 seed solid; crosses on the NA row are hops that never become decodable.

Table B.2: Skipping eight consecutive layers of the untied 32-layer model (matched-rate cohort): last-query accuracy at $H = 8$, seeds 42 / 43 / 44. Intact accuracy is 1.000 / 0.980 / 0.984.

Skipped layers	1–8	9–16	17–24	25–32
Accuracy	.082 / .000 / .098	.204 / .058 / .118	.160 / .142 / .106	.092 / .046 / .042

Readability and solving. Figure B.4 shows the peak selectivity of every reference hop next to first-query accuracy for the models probed at $H = 8$. In (a), looped seed 44 solves $H = 8$ with peaks of 0.28 to 0.56 on hops 2 to 7, and the looped run without the curriculum peaks at 0.24 or below on hops 1 to 7. At $H = 15$, where hop h is $16 - h$ backward hops, that run decodes only hop 15 (one backward hop, layer 7) and hop 14 (two backward hops, layer 8).

Training without iteration embeddings. Table B.3 gives the looped 4×8 model trained with τ_t removed, so every iteration receives exactly the same parameters and no index. No untied model solves at this learning rate, so the control has no untied counterpart.

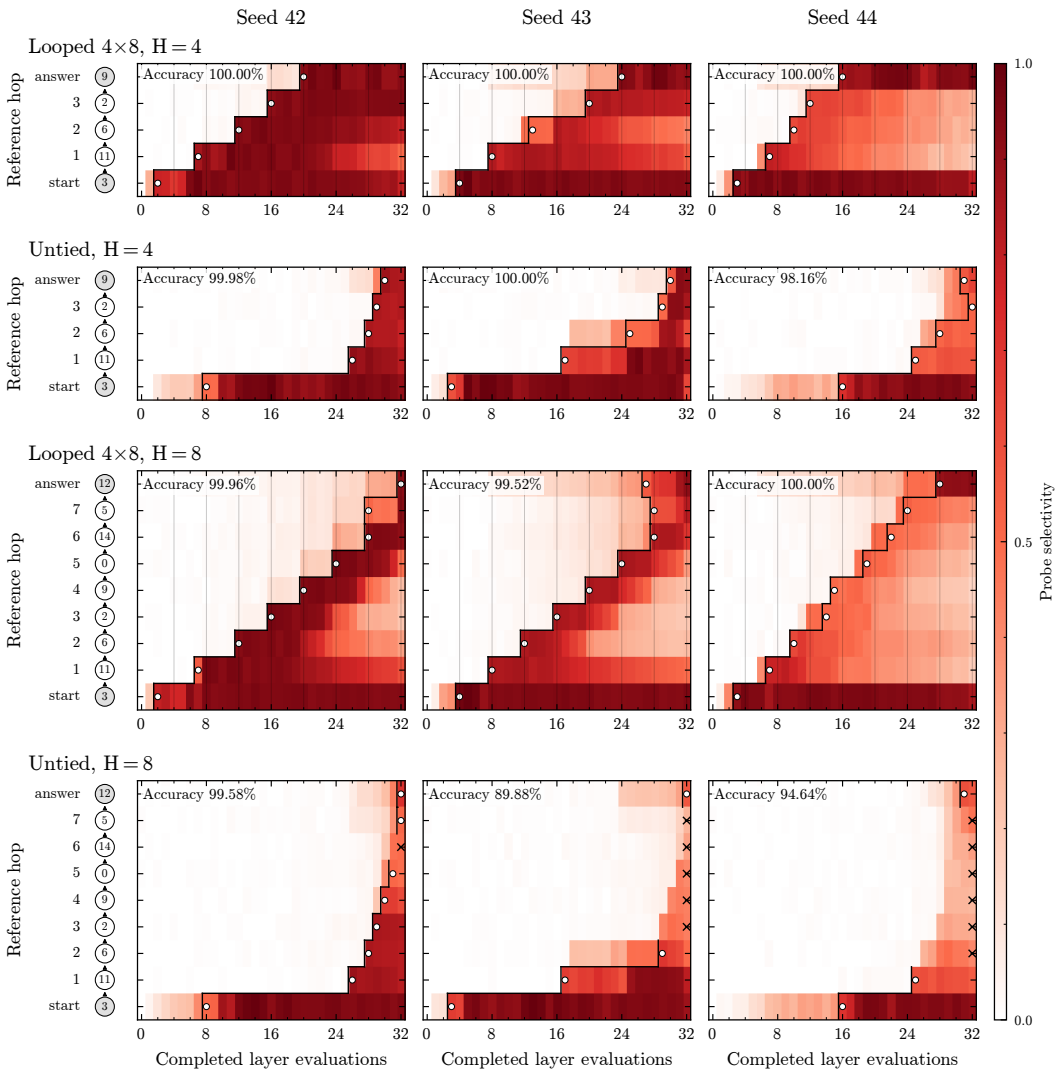


Figure B.2: **All matched checkpoints.** Probe selectivity maps for the looped 4×8 and untied 32-layer models of the matched-rate cohort, seeds 42 to 44, at $H = 4$ and $H = 8$, drawn as in Figure B.1. White dots and black lines mark onsets at selectivity 0.5, crosses mark hops that never cross, and faint grey lines mark block boundaries. Accuracy is first-query accuracy on 5,000 prompts.

Table B.3: Looped 4×8 trained without iteration embeddings (learning-rate 10^{-3} cohort): first-query accuracy (%) and onsets at selectivity 0.5 in layer evaluations. Seeds 42 to 44 reach a solved prefix of 15 (at 56k, 68k and 80k steps); seed 45 ends at 9.

Seed	$H = 4$		$H = 8$	
	Acc.	Onsets	Acc.	Onsets
42	99.94	9, 12, 14, 22	91.72	9, 12, 14, 22, 26, 28, 28, 30
43	100.00	5, 10, 13, 18	99.86	5, 10, 13, 18, 21, 26, 29, 32
44	100.00	8, 9, 12, 16	100.00	8, 9, 12, 16, 20, 24, 28, 32
45	19.44	7, 8, –, –	77.84	7, 8, –, –, –, –, –, 32

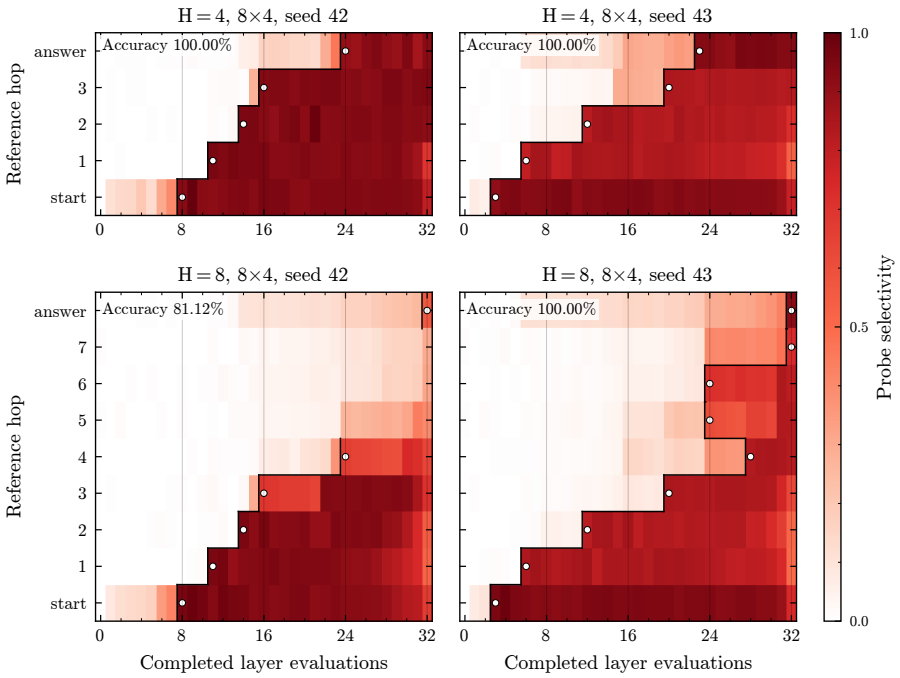


Figure B.3: **An eight-layer block applied four times** (8×4). Probe selectivity maps at $H = 4$ (top) and $H = 8$ (bottom) for seeds 42 and 43 of the shared-initialisation cohort, drawn as in Figure B.2. Accuracy is first-query accuracy.

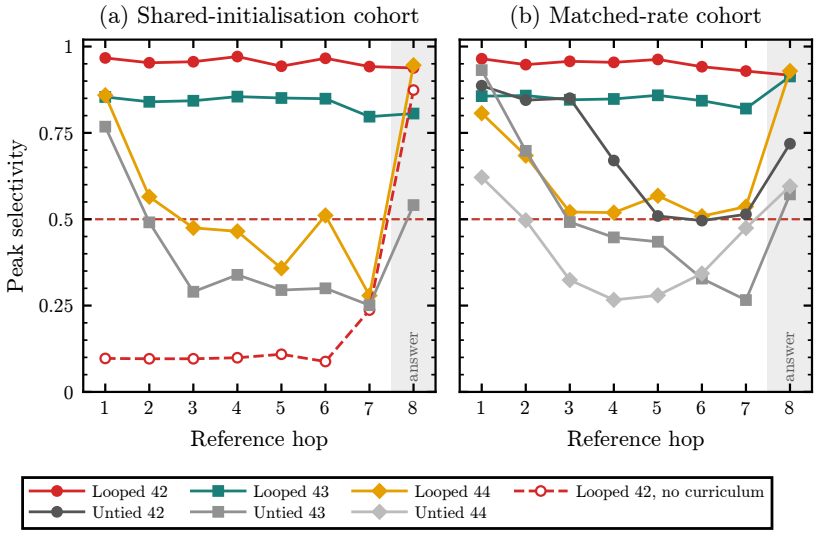


Figure B.4: **Readability and solving at $H = 8$.** Peak selectivity over depth for each reference hop at 100k steps (hop 8 is the answer) in (a) the shared-initialisation cohort, trained for this paper, and (b) the matched-rate cohort ($\eta = 3 \times 10^{-4}$); the dashed red line marks 0.5. First-query accuracy for looped seeds 42, 43, 44 is 1.000, 0.990, 1.000 in (a) and 1.000, 0.995, 1.000 in (b); untied seed 43 scores 0.923 in (a), where untied seed 42 (0.160, unsolved) is not drawn, and 0.996, 0.899, 0.946 in (b); the looped run without curriculum scores 0.998.

State transfer. To test whether the probed states are used, we overwrite the receiver's state at the read position with that of a donor prompt with the same start node and a different cycle, at the onset of hop j and the two neighbouring layers (protocol in Appendix A.6). A model that reads the transplanted state as its current node finishes the traversal on the receiver's cycle from the donor's node, an answer correct for neither prompt. Table B.4 counts these composed answers in every trial window.

Table B.4: State transfer in the matched-rate checkpoints at $H = 4$: composed answers out of 128 trials at each of the three depths of hop j, in depth order. *Node*: node-subspace edit; *Whole*: whole-state transfer; *Alt.*: same-node alternative donor; *Shuf.*: shuffled-node donor scored on the primary target, and *Own* on its own target. Unpatched, identity and orthogonal edits give 0 in all 52 windows. Untied seed 44 has one valid depth for hop 3 because its onset is the last layer.

	Seed	j	Depths	Node	Whole	Alt.	Shuf.	Own
Looped	42	1	6–8	0/0/69	0/0/81	0/0/71	0/0/2	0/0/70
		2	11–13	0/58/65	2/78/79	0/57/65	0/0/0	0/60/67
		3	15–17	0/53/50	3/69/69	0/54/45	0/4/4	0/66/61
	43	1	7–9	0/20/11	0/82/82	0/16/11	0/3/0	0/10/11
		2	12–14	0/1/1	44/44/44	0/0/0	0/0/0	1/1/3
		3	19–21	0/14/17	29/58/58	1/20/16	0/1/1	0/19/20
	44	1	6–8	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
		2	9–11	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
		3	11–13	0/0/0	0/0/0	0/0/0	0/0/0	0/0/0
Untied	42	1	25–27	0/0/50	0/3/81	0/0/56	0/0/2	0/0/68
		2	27–29	0/36/0	4/78/5	0/36/0	0/0/0	0/41/0
		3	28–30	0/1/0	3/58/11	0/0/0	0/1/0	0/4/0
	43	1	16–18	0/13/9	0/61/35	0/12/9	0/3/2	0/6/7
		2	24–26	0/0/0	2/42/42	0/0/0	0/0/0	0/0/0
		3	28–30	0/0/1	3/43/29	0/0/0	0/0/0	0/0/0
	44	1	24–26	0/0/0	0/0/0	0/0/0	0/0/1	0/0/0
		2	27–29	0/0/0	7/7/8	0/0/0	0/0/0	0/0/0
		3	31	0	7	0	0	0

B.2 Partial pooling and gradient routing (§4.1 and §4.2)

Table B.5 gives the final values behind Figures 1 and 5 for every seed. Within a seed, the partial-pooling arm with the larger α solves first in 24 of the 30 pairs of arms and later in 3, with 3 ties between arms that never solve (one-sided permutation test over the α labels within each seed, $p < 0.001$). At $H = 4$, the more pooled arm of a pair decodes hop 1 earlier in 29 of the 30 within-seed pairs, with one tie and no reversals (exact one-sided permutation test, $p = 1.2 \times 10^{-6}$), and the mean onset over hops 1 to 4 orders the same way in 27 of 30 ($p = 8.8 \times 10^{-5}$).

Table B.5: Gradient routing and partial pooling at 100k steps, seeds 42 / 43 / 44: last-query accuracy at $H = 8$ (§4.1 quotes first-query accuracy on the probe prompts, which differs slightly), solved prefix (Appendix A.4) and the first evaluation at which every hop length up to $H = 8$ is solved (thousands of steps; a dash means never).

Block update	$H = 8$ accuracy	Solved prefix	Solves $H \leq 8$ at (k)
(i) $G = \sum_t g_t$	.998 / .994 / 1.000	15 / 15 / 15	24 / 68 / 36
(ii) $8g_8$	.390 / .385 / .379	1 / 2 / 1	– / – / –
(iii) $\|G\|g_8/\|g_8\|$	.535 / .396 / .373	1 / 1 / 1	– / – / –
(iv) $8g_J$, J uniform	.317 / .139 / .176	0 / 0 / 6	– / – / –
$\alpha = 1.00$	.987 / 1.000 / 1.000	15 / 15 / 15	32 / 48 / 32
$\alpha = 0.75$	.803 / .989 / .931	15 / 15 / 15	96 / 56 / 72
$\alpha = 0.50$	.589 / .966 / .824	5 / 15 / 15	– / 68 / 92
$\alpha = 0.25$	.807 / .989 / .994	6 / 15 / 15	– / 60 / 56
$\alpha = 0.00$	.557 / .741 / .730	5 / 6 / 4	– / – / –

B.3 Formation and within-block localisation (§4.3)

Figure B.5 repeats Figure 6 for the second seed. Figure B.6 folds the 32 layer evaluations of the looped 4×8 model onto the four layers of its block. The untied onsets of Table B.1 fall at every position modulo four (seed 42, $H = 4$: layers 26, 28, 29, 30), and removing any eight-layer window of the untied model is damaging (Table B.2).

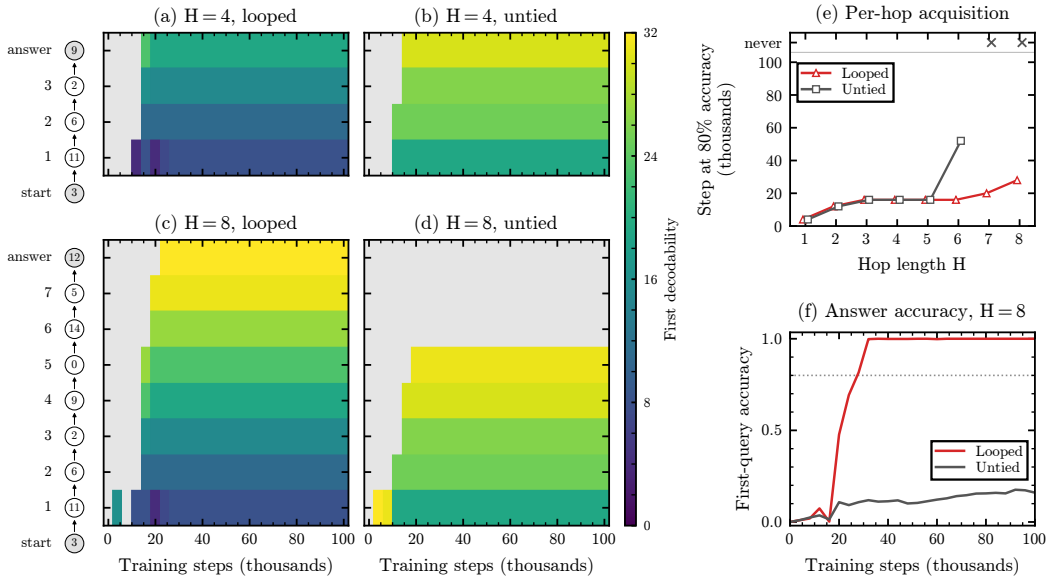


Figure B.5: **Staircase formation during training, seed 42** of the shared-initialisation cohort, drawn as in Figure 6.

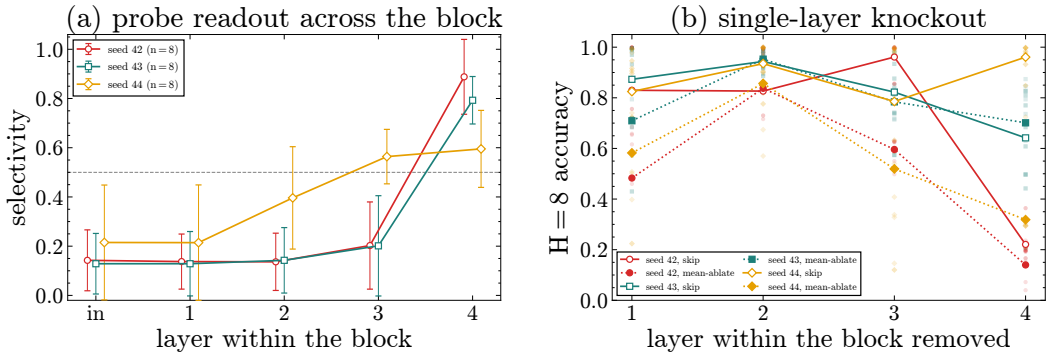


Figure B.6: **Inside one block of the looped model**, matched-rate cohort, $H = 8$, three seeds, folded onto the four layers of the shared block. (a) For each hop, selectivity across the block in which it first crosses 0.5, from the block input (in) through its four layers; points are the per-seed mean over the eight hops, error bars one standard deviation across hops, seeds offset slightly in x. (b) Last-query accuracy after removing one layer at one iteration, folded by position in the block and averaged over the eight iterations (small markers: individual iterations); solid is skipping the layer, dotted is replacing its contribution by a reference mean.

B.4 Sharing partitions (§4.4)

Table B.6 gives the per-seed values behind Figure 7; panel (c) of that figure uses seeds 42 and 43, the two seeds every partition has.

Table B.6: Sharing partitions of the shared-initialisation cohort at 100k steps: onsets at selectivity 0.5 (a dash marks no crossing) and the first evaluation at which first-query $H = 8$ accuracy reaches 0.8 (thousands of steps; a dash means not within 100k). The untied partition has seeds 42 and 43 only.

Partition	Seed	$H = 4$ onsets	$H = 8$ onsets	0.8 at (k)
Looped AAAAAAAA	42	8, 11, 15, 19	8, 11, 15, 19, 23, 27, 31, 32	28
	43	8, 12, 16, 20	8, 12, 16, 20, 24, 28, 32, 31	52
	44	6, –, 12, 16	6, 11, –, –, –, 20, –, 28	36
Cyclic ABABABAB	42	11, 19, 27, 31	11, 19, 27, –, –, –, –, 29	44
	43	12, 16, 21, 24	12, 16, 21, –, –, –, –, 25	40
	44	7, 10, 15, 23	7, 11, 16, 23, –, –, –, 32	48
Contiguous AAAABBBB	42	20, 24, 28, 32	20, 24, 28, –, –, –, –, 32	96
	43	17, 21, 25, 27	17, 21, 25, 29, 30, –, –, 32	52
	44	17, 21, 25, 29	17, 21, 25, 29, –, –, –, 31	52
Untied ABCDEFGH	42	19, 25, 26, 30	19, 25, 26, 30, 31, –, –, –	–
	43	22, 26, 29, 30	22, –, –, –, –, –, –, 32	84

C A LARGER GRAPH AND A SECOND TASK

$N = 32$. The matched pair was trained on 32-node cycles at learning rate 3×10^{-4} with context 384 and the recipe otherwise unchanged. The looped model reaches solved prefixes of 6, 1 and 6 in three seeds, and the untied model 1 and 1 in two; some runs ended before 100k steps (last evaluations at 84k and 52k for two looped seeds and at 52k for one untied seed), and one further untied run with a truncation and evaluation fault is excluded. The optimization recipe was not recalibrated for $N = 32$.

Register machine. A prompt is a straight-line program of up to 15 arithmetic and conditional-move instructions over four registers, followed by queries for the final value of one register; its vocabulary has 24 tokens. Both families, trained in the learning-rate 10^{-3} cohort, reach a solved prefix of 15 in all three seeds (two runs last evaluated at 84k steps). Program length is a weak measure of dependence depth on this task: the backward slice of the queried register, the instructions whose written values flow into the answer, averages 1.2 instructions at program length 4, 3.0 at 8 and 6.5 at 15. Linear probes at the query position of all six checkpoints, fitted on each query's own backward slice at program lengths 4 and 8, read the answer (peak selectivity 0.80 to 0.96), while the value one instruction back reaches 0.5 in one looped and four untied runs, never before the answer, and values further back stay below 0.5 in every run.